

A New Hope for DARPA OpTC

Frédéric Majorczyk, Barbara Pilastre, Fanny Dijoud

▶ **To cite this version:**

Frédéric Majorczyk, Barbara Pilastre, Fanny Dijoud. A New Hope for DARPA OpTC. CSET 2025 - Cyber Security Experimentation and Test at ACSAC 2025 International Workshops, Dec 2025, Honolulu, United States. ⟨hal-05474126v2⟩

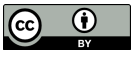
HAL Id: hal-05474126

https://inria.hal.science/hal-05474126v2

Submitted on 31 Mar 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

A New Hope for DARPA OpTC

1st Frédéric Majorczyk§ *0009-0008-9558-397X*
2nd Barbara Pilastre§ *0009-0006-0622-0575*
3rd Fanny Dijoud§ *0009-0007-2705-2977*

***Abstract*—This article addresses the challenges of using the DARPA OpTC dataset for intrusion detection system (IDS) research. While the dataset offers a rich combination of network and host data, its usability is hindered by a lack of an official event labeling and errors in logs data. We propose a two-fold solution: first, we identify and correct errors in the dataset, and second, we design and implement a comprehensive labeling methodology for attack-related events at both the network and host levels. Our corrected dataset, along with the labeling scripts, will be made publicly available to support reproducibility and further research. Additionally, we assess the impact of these labels and corrections on the effectiveness of graph-based machine learning IDS methods.**

***Index Terms*—Intrusion Detection, Dataset**

1. Introduction

Intrusion detection systems (IDS) are widely used to defend against cybersecurity threats. IDS detect attacks by monitoring network, system, or application activities, enabling a quick response from security teams to mitigate threats. Meantime cyberattacks are becoming more frequent and sophisticated, requiring IDS to evolve for effective detection. Thus interest in AI and machine learning-based IDS has increased. To evaluate these new IDS, researchers often rely on public datasets containing normal and malicious activities. Many datasets have been published for network intrusion detection [1]–[6] in the last 25 years while only a few have been published for host intrusion detection [7]–[11]. Even fewer datasets contain both network and host data [12]–[14].

Among those datasets, the DARPA OpTC (Operationally Transparent Cyber) dataset [14] is an interesting one, mainly for three reasons: (1) it contains network and host data allowing the combination of those data to detect intrusions, (2) the information system used to generate the data is large (1000 computers) resembling a real IT system, and (3) Red Team attacks are more sophisticated, mimicking APT scenarios, and harder to detect than common attacks like DoS or Brute Force found in other datasets. The DARPA OpTC dataset is thus more and more used by the research community. Even if it contains network data, most work that exploits it [15]–[18] only uses the host data and constructs provenance graphs [19].

Despite its advantages, the DARPA OpTC dataset has drawbacks limiting its use. First, there is no official labeling of attack-related events at the network or host level. The only official available information is a text document describing the Red Team's actions, detailing compromised computers and `pid` of agents. Consequently, research projects infer their own labels, often without publishing or explaining the process, making it difficult to reproduce results or compare work. Even when labels are published, the lack of explanation raises concerns about their accuracy. In 2024, Nikulshin and Talhi [20] published a labeling of the dataset, which they updated in 2025. Additionally, we identified evidence of out-of-order reception of system events by the logging system, resulting in incorrect process tracking. Specifically, we observed mismatches and overlaps in unique identifiers associated with processes, files, and network flows. These inconsistencies compromise the integrity of provenance graphs and can introduce noise or bias into IDS training and experimental evaluations. These errors also complicate manual alert analysis, leading to confusion about the objects.

Since the DARPA OpTC dataset is one of the most realistic publicly available dataset for intrusion detection research, it serves as a valuable benchmark for the development, evaluation, and comparison of IDS techniques. This work aims to provide the research community with a cleaned and accurately labeled version of the dataset to support these efforts. The contributions of this paper are twofold: (1) to correct errors in the dataset and evaluate the impact of these corrections on the performance of state-of-the-art IDS; and (2) to propose a labeling of attack-related events at both the network and host levels, which we compare to the labeling proposed by Nikulshin and Talhi. The corrected dataset and all scripts used for correction and labeling are available in a GitLab repository[1] and on a dedicated web page[2]. Additionally, we evaluate the impact of these corrections on the detection performance of state-of-the-art IDS graph-based machine learning methods.

The article is organized as follows: Section 2 details the DARPA OpTC dataset and its errors. Section 3 describes our correction methodology and provides a statistical analysis of the errors. Section 4 explains the labeling process and compare our labels with the ones of Nikulshin and Talhi.

§. Barbara Pilastre contributed to the corrections of the logs, Frédéric Majorczyk to the labeling of the logs and Fanny Dijoud to the experiments.

1. https://gitlab.inria.fr/fmajorcz/a_new_hope_for_darpa_optc.git
2. https://correctedoptc.inria.fr/

Section 5 presents experiments measuring the impact of labels and corrections on detection results. Section 6 offers insights on creating datasets for intrusion detection, and Section 7 concludes the article.

2. DARPA OpTC: presentation and errors

In this section, first, we present the dataset more in details using information found on the website and evidences that we found by analyzing the logs and then we explain the errors we found in the logs.

2.1. DARPA OpTC in details

The dataset covers a nine-day period from the 16th to the 25th September 2019. The first six days contain only benign activities. During the last three days, three different and independent APT attacks were launched against the IT system (one for each day). The IT system consists of one thousand Windows 10 client computers and one active directory (called DC1). It seems to be connected to the Internet (or a fake Internet) via at least one other router as some of the APT attacks use specific domain names (such sports.com or microsoft.com). There are no precise information on how the benign activity was generated but we found traces of scripts launched on each client computer. They may simulate user activities by browsing the Internet, editing documents, etc.

The dataset consists of two types of logs: logs from a Bro NIDS (called Zeek now) and logs from the Windows 10 client computers. The description of the dataset does not provide any information on the location of the Bro NIDS. However, as these logs contain only information about connections between computers inside the IT system and the Internet (i.e., there is no information about connections between clients themselves or between clients and the DC1), the most likely hypothesis is that the Bro NIDS captured only traffic between the IT system and the external network.

About the Windows 10 client logs, the dataset description asserts that only the logs of 500 computers are given but, in fact, depending on the day, logs from more than 500 computers are provided. These logs are low-level logs resembling to Sysmon logs. There are no information on the tool used to generate those logs and the information about the format of the logs is very scarce[3]. The format is an extension of the CAR model. It consists of a tuple with three main components: `Actor`, `Action` and `Object`. The `Actor` represents a process that executes the `Action` (`WRITE`, `OPEN`, etc.) on the `Object` (`FILE`, `REGISTRY`, etc.). Other fields provide complementary information about the event (`timestamp`, `pid`, etc.). The description of the format provides neither the syntax nor the vocabulary used in the logs. The work of Anjum et al. [21] gives details about the vocabulary that can be possibly used in those logs by analyzing the distribution of values for the different fields. As an example, Listing 1 is an eCAR log indicating that the process of pid `4772` on `SysClient0063.systemia.com` terminates at timestamp `2019-09-20T23:15:42.194-04:00`. This process was an instance of the `conhost.exe` executable launched by the user `agilbard`. We also notice a unique identifier for the event `id`, for the process executing the action `actorID` and for the object on which the action is taken `objectID` (here the object is the process itself as the log represents the termination of the process).

Listing 1. eCAR log of the termination of a process


```
{
 "action":"TERMINATE",
 "actorID":"5689402e-9c41-4fe1-aff4-a0b43a8417cf",
 "hostname":"SysClient0063.systemia.com",
 "id":"71ac43d3-3bb8-44b9-9bad-127ddd0c57fe",
 "object":"PROCESS",
 "objectID":"5689402e-9c41-4fe1-aff4-a0b43a8417cf",
 "pid":4772,
 "ppid":776,
 "principal":"SYSTEMIACOM\\agilbard",
 "properties":{
  "command_line":"\\??\\C:\\Windows\\system32\\conhost.
      exe 0xffffffff -ForceV1",
  "image_path":"\\Device\\HarddiskVolume1\\Windows\\
      system32\\conhost.exe",
  "parent_image_path":"\\Device\\HarddiskVolume1\\
      Windows\\system32\\conhost.exe",
  "sid":"S-1-5-21-4190936083-3304963419-
      1584388968-3208",
  "user":"SYSTEMIACOM\\agilbard"
 },
 "timestamp":"2019-09-20T23:15:42.194-04:00"
}
```


There is one design choice in the logs that is debatable: the `pid`, `image_path` and `command_line` refer to the process that performs the action and that is identified by the `actorID`, except when the `action` is `CREATE` and the `object` is `PROCESS`. In that particular case, the `pid`, `image_path` and `command_line` refer to the process that is created and identified by the `objectID`. A consistent matching between these fields and the `actorID` should have been preferable.

2.2. Errors in eCAR logs

Our initial exploratory data analysis of the dataset confirms the findings of Anjum et al. [21], as we observed temporarily unordered logs, which appears to result from asynchronous reception by the monitoring system. This introduces numerous inconsistencies in the eCAR logs, such as those illustrated in the following listings.

About 4 hours after the termination presented in Listing 1, a new process with the `id` `"b8d71d4a-2184-4217-8383-123f66043e39"` is created reusing the `pid 4772` that was freed by the previous termination. Listing 2 presents the creation of this process. Less than 1 second after this creation, there is the termination of a process with the same `pid 4772` but with the `id` used in the first termination (Listing 3). Three things seems inconsistent at this point: the `pid 4772` should be associated with the process created in Listing 2

3. https://github.com/FiveDirections/OpTC-data/blob/master/ecar.md

as it is the last process created using this pid, the process of id 5689402e-9c41-4fe1-aff4-a0b43a8417cf is already terminated so this id should not appear in an event 4 hours after its termination, and, finally, some properties of the event presented in Listing 3 are not consistent. The values of the sid and the user are indeed not the same as the ones previously associated with that process. However, they correspond to the values of the process newly created (Listing 2). The eCAR log for the second termination appears to mix information from the previous process with the id "5689402e-9c41-4fe1-aff4-a0b43a8417cf" and the newly created process with the id "b8d71d4a-2184-4217-8383-123f66043e39".

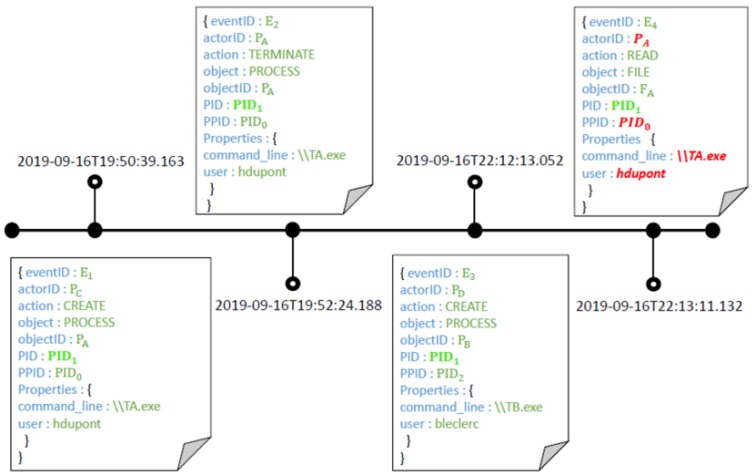
What corroborates this analysis is the fact that the logs in the original log file are not sorted by timestamp: the logs that are related to the pid 4772 that mix information (such as the one in Listing 3) are stored before the log of the creation of the process (Listing 2) (even if the termination happens after the creation). It seems the log management system affects to an eCAR log the last known values of the id and of some other fields, that are associated with a given pid, when the log is stored.

Listing 2. eCAR log of the creation of a process

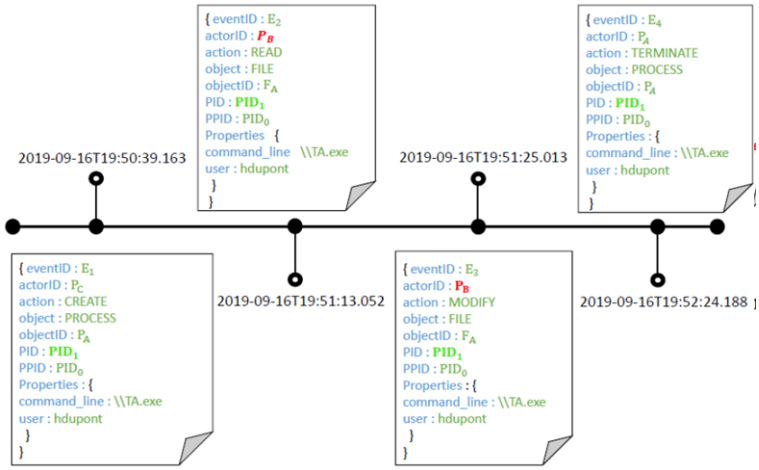

```
{
 "action":"CREATE",
 "actorID":"ecb11af3-3262-4fae-bb3c-083fc9475443",
 "hostname":"SysClient0063.systemia.com",
 "id":"f946815a-d428-41c5-8a5f-d808d0c9bd45",
 "object":"PROCESS",
 "objectID":"b8d71d4a-2184-4217-8383-123f66043e39",
 "pid":4772,
 "ppid":3812,
 "principal":"NT AUTHORITY\\SYSTEM",
 "properties":{
  "command_line":"C:\\Windows\\system32\\cmd.exe /c "
      schtasks /delete /tn "KickoffAutoLoginRandom" /F
      "",
  "image_path":"\\Device\\HarddiskVolume1\\Windows\\
      system32\\cmd.exe",
  "parent_image_path":"\\Device\\HarddiskVolume1\\
      Python27\\python.EXE",
  "sid":"S-1-5-18",
  "user":"NT AUTHORITY\\SYSTEM"
 },
 "timestamp":"2019-09-21T03:03:07.087-04:00"
}
```


Listing 3. eCAR log of a second termination


```
{
 "action":"TERMINATE",
 "actorID":"5689402e-9c41-4fe1-aff4-a0b43a8417cf",
 "hostname":"SysClient0063.systemia.com",
 "id":"15aff6c7-a663-4244-a97a-258fc66625ed",
 "object":"PROCESS",
 "objectID":"5689402e-9c41-4fe1-aff4-a0b43a8417cf",
 "pid":4772,
 "ppid":776,
 "principal":"SYSTEMIACOM\\agilbard",
 "properties":{
  "command_line":"\\??\\C:\\Windows\\system32\\conhost.
      exe 0xffffffff -ForceV1",
  "image_path":"\\Device\\HarddiskVolume1\\Windows\\
      system32\\conhost.exe",
  "parent_image_path":"\\Device\\HarddiskVolume1\\
      Windows\\system32\\conhost.exe",
  "sid":"S-1-5-18",
  "user":"NT AUTHORITY\\SYSTEM"
 },
 "timestamp":"2019-09-21T03:03:07.132-04:00"
}
```

(a) Timeline 1: use of an actorID after the termination of the process



(b) Timeline 2: use of two actorIDs for the same process

Figure 1. Examples of the of two identified error timelines.

The previous error is one of the two recurrent errors identified in the data and represented as timeline errors in Figure 1. The first error (Figure 1(a)) is similar to the previous example: some events are associated with an actorID that is already terminated. In the example of Figure 1(a), the process P_A is created (by process P_C) at timestamp 2019-09-16T19:50:39.163, which is reported by the event E_1 and is terminated at timestamp 2019-09-16T19:52:24.188 reported by the event E_2. However, the actorID of this process is once again used at timestamp 2019-09-16T22:13:11.132 (event E_4), which should not be possible since the process was terminated before. In the timeline, it should be noted that there

is also the creation of another process (P_B) with the same `pid` before the event E_4 (event E_3). In this timeline, we think that the `actorID` of event E_4 should be P_B and the different fields `ppid`, `command_line` and `user` should be the same as the ones of event E_3.

The second kind of errors relies on the use of two unique `actorIDs` for the same `pid` at the same time. Indeed, as described in Figure 1(b), the process P_A with the `pid` PID_1 is created at timestamp `2019-09-16T19:50:39.163` (event E_1) and is terminated at timestamp `2019-09-16T19:52:24.188` (event E_4). During this time span, another active process, P_B, is also associated with PID_1, which is not possible since a `pid` should be associated with only one `actorID` at a time. In this timeline, the creation and termination of process P_B are not observed in the logs. To fix such errors, we consider that `actorID` P_B should be replaced by P_A in events E_2 and E_3, as P_B has no legitimate reason to exist or to have been created according to the system logs.

These two kinds of error affect about 40% of processes and 4.22% events of the eCAR logs (see Table 1). Regardless of the method used, intrusion detection strongly relies on the quality of the data. These errors can thus skew training for machine learning-based IDS and experimental results for all IDS. Furthermore, the errors make it difficult for cyber experts to analyze alerts raised by an IDS by mixing information in the logs. Moreover, as these errors seem to follow patterns, we develop a methodology to fix them and provide a corrected dataset to the community.

3. Corrections of the eCAR logs

3.1. Correction methodology

The proposed correction of the DARPA OpTC eCAR logs focuses on the two main errors described below. The correction of the first type of error (timeline 1) aims to detect the use of terminated processes and correct involved events by replacing the information of the late process by the last created process. In Figure 1(a), E_4 is detected as an erroneous event as the `actorID` was terminated before (event E_2). For that type of event (identified by the values of `action` and `object`), we replace the values of the `actorID`, `ppid`, `command_line` and `user` by the values found in the creation of process P_B (event E_3), since P_B has the same `pid` and has been created few seconds before the erroneous event. Depending on the event type, some other field values have to be replaced too (`image_path`, `objectID`, `tgt_pid_uuid` or `task_process_uuid` for example). The correction of the second case supposes to identify 'usurper' processes, i.e. the processes that appear spontaneously and short-circuit certain legitimate processes. In Figure 1(b), process P_B is an usurper process since it shares the same `pid` of process P_A during P_A's lifespan and is associated with any creation and terminate events. Thus, events E_2 and E_3 linked to P_B are erroneous. For this error type, we assume that P_B did not exist and replace P_B with P_A in the concerned events.

3.2. Analysis of errors and corrections

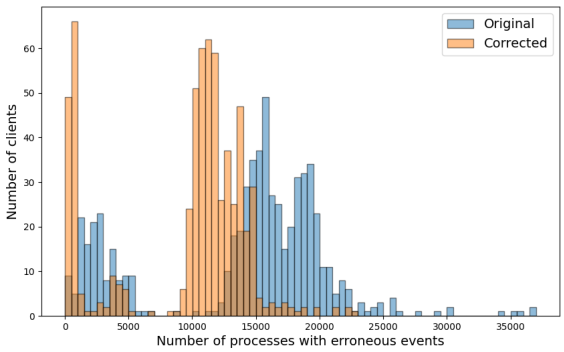
The OpTC dataset is extremely large, making its correction a time-consuming task. Multiple correction passes are required, as resolving earlier inconsistencies can reveal new ones. Successive passes thus ensure that all interdependent errors are progressively identified and corrected until timelines are fully consistent (see Appendix A for an example). As shown in Table 1, three correction passes were performed—a compromise between the time required and the diminishing effective correction. Initially, 4.22% of the dataset—about 700 billion events across more than 8.5 billion processes (40% of all processes)—contained errors. After three correction passes, the number of erroneous events was reduced by a factor of 2.5, correcting over 400 billion events and lowering the error rate to 1.6%.

TABLE 1. NUMBER OF ERRORS (PROPORTION IN PARENTHESIS)

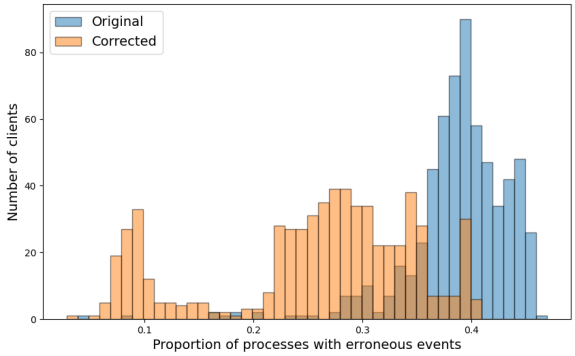
Correction	Processes	Events
Original	8 645 600 (40%)	701 039 318 (4.22%)
First pass	6 169 386 (28.83%)	313 063 564 (1.89%)
Second pass	6 035 310 (28.32%)	289 904 664 (1.75%)
Third pass	6 014 764 (28.24%)	261 603 670 (1.6%)

Figure 2 shows the distribution of errors by client, in terms of affected processes and erroneous events, providing a clearer analysis of the correction's effectiveness. Our main findings are that all clients are affected, though not equally, and the correction significantly reduces errors. Before correction, most clients had between 35% and 45% of processes with errors and 2% to 6% of erroneous events.After correction, error rates decreased markedly, with fewer than 35% of processes and 3% of events remaining affected. The correction also altered the error distribution, shifting the mode toward zero and indicating a lower error incidence per client. Nevertheless, approximately 30% of processes still exhibit errors, although only a few events within each process are now erroneous.

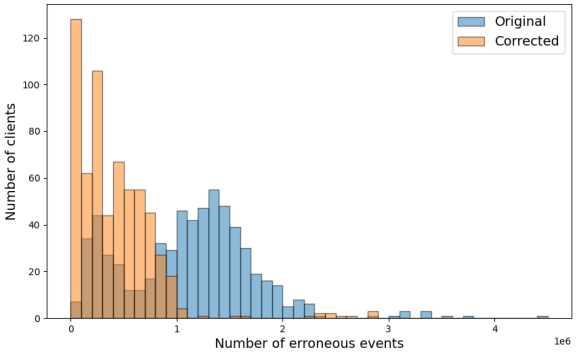
The eCAR logs are often used to construct provenance graphs used for intrusion detection. The corrections have an impact on the provenance graphs that are built. Figure 3 presents two graphs representing the activity of client 051 during 7 hours on the 25th of September. The first one is built using the original eCAR logs and the second one the corrected eCAR logs. The graphs are focused on two processes whose `actorIDs` begin by `241f978c` and `ffdd2745` that have the same `pid` `4600` at different times. The number on the edges represents the timestamp of the event. On the first graph, we can see that the activity of the process `ffdd2745` is limited to its creation at timestamp 21785. The other process `241f978c` creates many threads and opens many files at timestamps around 21790 (the upper part of the graph) while the beginning of its activity is around timestamp 361. Its termination (not shown on the graph) is also at timestamp 361. All the events at timestamps around 21790 are erroneously affected to process `241f978c`. After correction, these events are affected to the second process deeply changing the provenance graph.



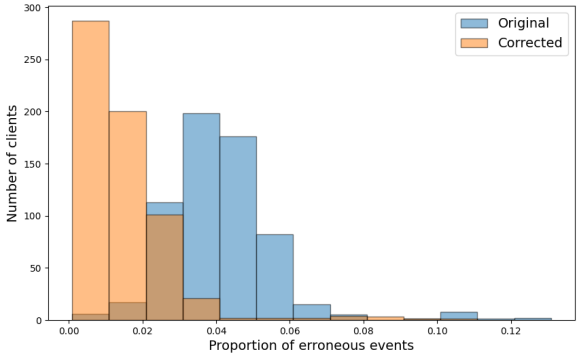
(a)



(b)

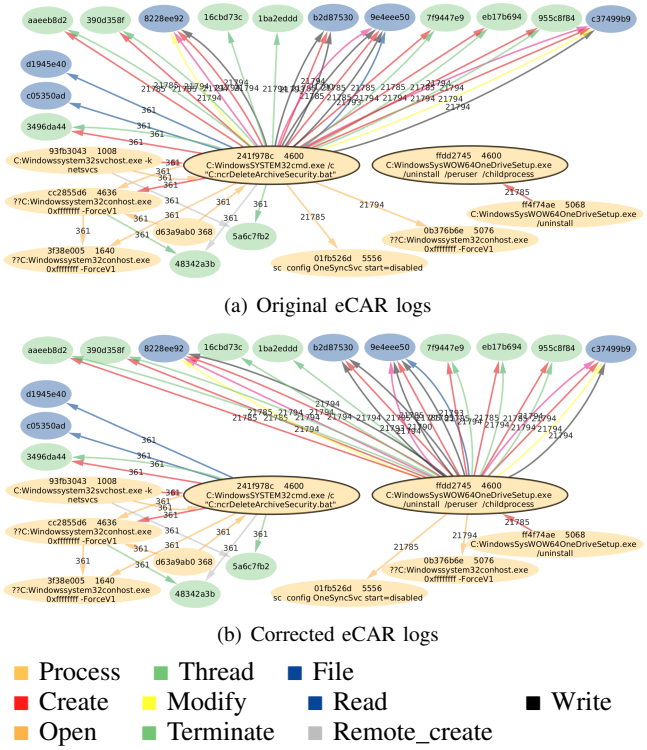


(c)



(d)

Figure 2. Distribution of errors before and after three correction passes.



(a) Original eCAR logs

(b) Corrected eCAR logs

Process Thread File
Create Modify Read Write
Open Terminate Remote_create

Figure 3. 1-hop neighborhood of two processes, involved in a Timeline 1.

3.3. Limits of the corrections

During our analysis, we encountered other errors, which were quite difficult to understand and decided not to correct all of them. First, we chose not to correct errors related to the `pid` 4 process, which is the System process in Windows. This process, launched first after boot, should have a unique `actorID` until the next reboot. However, we observed multiple `actorIDs` during reboot, when only two should appear. Due to difficulty in understanding the reboot logs and pinpointing the errors, we decided not to correct the `actorID` or other fields for the System process.

Secondly, we noted some problems with the timestamps of some eCAR logs. For example, there were modifications to registry keys made by a process three minutes before its creation. We could have changed the `timestamps` of those events so that they happened during the lifespan of the process but it is not possible to determine precisely the new `timestamps` of those logs.

4. Labeling of DARPA OpTC

This section explains how we labeled the dataset at the host and network level and compares our host labeling with the one of Nikulshin and Talhi [20]. Finally, we discuss some choices made during the labeling.

4.1. Labeling approach

To label the dataset at the host and network level, we design one approach for each level and then use logs that give the matching between Bro id and event id (eCAR-Bro logs) to correlate and complete the labeling at the two levels. As input, we use the PDF file provided with the dataset, that summarizes each scenario, including details on the tools, configurations, attack infrastructure (e.g., C&C servers), and compromised computers with their IP addresses, `pids`, and IDs of some malicious agents.

At the network level, the approach is quite simple: the IP addresses and hostnames of the clients compromised and the IP addresses and/or hostnames of the C&C servers and websites used for the attacks are given. As the Bro logs do not contain internal communications, we are not able to label the lateral movements of each scenario. Moreover, the timestamps of some network actions (website visited, agent check-in, termination of an agent, RDP sessions) are given. The first step is to filter all communications based on those IP addresses (network flows between a client or DC1 and a C&C server). As the timestamps in the PDF file may not be precise, we filter the communications on each day (except for the Scenario 2 because the agents communicate with the C&C server over night). We then manually check the different communications by analyzing all the Bro logs available for each unique bro ID. This step is necessary as we are not sure that all communications between the C&C servers and the clients are malicious, especially because the C&C servers use some common DNS names (sports.com or news.com). We also check for other computers connecting to the C&C servers of each scenario. For the first scenario, we found four computers that communicate with the IP address of the C&C server and that are not listed in the PDF file. As the patterns of connections (URL accessed, delay between connections, etc.) is the same as for the compromised ones, we decided to label those connections as malicious too. Unfortunately, the eCAR logs of those four computers are not provided so it is impossible to correlate with the host activity. For the second and third scenario, we also found one computer connecting to the C&C server and not listed in the PDF file for each scenario: the patterns and features of connections are similar to the malicious ones. The eCAR logs of those computers are not provided. We decided to label those connections as malicious, nonetheless.

At the host level, for the eCAR logs, the approach is a bit more complicated. The PDF file only provides the `pids` of some malicious agents (and for Scenario 3, the `pid` of the processes in which the malicious agent inject themselves). We decide to label all the events where the `actorID` is a malicious agents as malicious. So the labeling is made at the level of processes. We also make the hypothesis that all the children of a malicious process are malicious. This implies that it is necessary to follow the creations and terminations of processes as we did for the corrections of the errors in the eCAR logs. For all the agents listed in the PDF file, we also checked manually whether their parent is malicious or not. We also manually inspected the logs to find the pid of agents listed without a pid in the PDF file. We did the same thing to identify the malicious processes for the RDP sessions of Scenarios 2 and 3. After having identified the pid of processes linked to the activity described in the PDF file, we just follow the creations and terminations of those processes, label their children and finally all the events made by those processes as malicious.

The last step of our labeling is to use the eCAR-Bro logs provided with the dataset to check that a network connection linked to a malicious process is also labeled as malicious and conversely, that a process that makes a network connection labeled as malicious is also labeled as malicious.

4.2. Comparison with the labeling of Nikulshin and Talhi

Compared to the work of Nikulshin and Talhi, we also label the network connections, allowing us to correlate the Bro logs with the eCAR logs. For the eCAR logs, our approach is similar to theirs, but there are still some differences in the results. We label as malicious 360925 events while they label 313018 events. Regarding the number of `actorIDs`, we label 1103 `actorIDs` while they label 779. The main differences are due to the fact that: we label `notepad++.exe` as malicious since it contains the malicious executable that downloads the meterpreter in Scenario 3; we label the RDP connections differently; they label some anomalous activities on two hosts before the date of the first scenario (maybe due to some Red Team testing); they label some activities not described in the PDF file during the APT attacks; there are some timestamps errors in the PDF file; they do not always account for the errors in the `actorIDs` missing some events.

The two labelings of the eCAR logs are very close: this leads to a good level of confidence for the correctness of these two set of labels.

4.3. Discussion

Labeling a dataset precisely is quite complicated and depending on the analyst, some parts of the labeling can be different. For example, for the RDP connections used by the Red Team, the user session is clearly malicious: it should not have occurred. However, if we decide to label `winlogon.exe` and all its children as malicious, many of the children processes are benign processes launched at the opening of each user session. These benign processes would be mislabeled as malicious. Another example is when the Red Team downloaded an executable on one client and the first execution of the executable is to print the help (with the `-h` option). This execution does not do any harmful actions on the system. However, the executable should not be on that client and thus should never have been executed.

The main issue with this grey area in the labeling is that it can impact the classical metrics (precision, recall, f1-score, etc.) computed to measure the performances of IDS and give some bad insights on the usefulness of an IDS.

5. Machine learning experiments on both versions of DARPA OpTC, using different labels

In this section, we evaluate the impacts of the corrections and the labels on the performance of three different state-of-the-art unsupervised provenance-based IDS: Kairos [17], Flash [22] and a Graph Auto-Encoder (GAE) [23] [24].

5.1. Graph representation

Provenance graphs [19] are directed multi-edges graphs where nodes represent objects of the operating system (e.g., process, file) and edges represent actions made by a source object on a sink object at a given time (e.g., create, read). These graphs highlight causal and temporal links, providing a clearer representation of system activity and revealing relationships between entities present in the logs.

For Flash and Kairos, we use their publicly available code. In contrast, for the GAE, which serves as an anomaly detection baseline, we need to define the features of the nodes and edges. We use both the system-defined object type (e.g., `PROCESS` or `FILE`) and the node's lifetime within the graph as node features. For the edges, we use the `action` field (e.g., `READ`) and the `timestamp` field from the eCAR logs as edge features. The `actorID` and `objectID` fields provided by the eCAR logs are used to uniquely identify each node. Following the approach of Kairos, we construct graphs that represent fifteen-minute intervals of system activity.

5.2. Descriptions of the experiments

We evaluate and compare the three different IDSs mentioned before. Flash [22] and Kairos [17] are unsupervised anomaly-based provenance-based IDS. One key difference between them is that Flash raises alerts about entities of the provenance graph, while Kairos raises alerts about a graph representing 15 minutes of activity. In our experiments, we trained the GAE with a link prediction objective, and used it to detect intrusion at the graph level.

The experiments focus on three critical clients, each involved in a distinct attack scenario and key stages of the intrusion. For each IDS, we train two models: one on the original eCAR logs and one on the corrected version, using benign activity from September 22. The IDS are treated as end-to-end IDS, applying the models to input data and presenting the results in Table 2 and Table 3, without optimizing the models. The performance evaluation is based on common metrics [25].

To compare the performances regarding the labels used (Table 2), we only use the original eCAR logs, as labels from Nikulshin and Talhi [20] and from Flash only refer to this version of logs. Flash's entity-level labels can not be applied to Kairos and GAE, as these approaches are at graph level. To compare the performances regarding the versions of the eCAR logs used (Table 3), we focus on our labels.

5.3. Experiments' result

The results in Table 2 show clear performance differences across the three IDSs and the impact of the labeling. Based on the outputs from Flash and Kairos, our labeling and that of [20] yield nearly equivalent results, suggesting strong similarity. However, these findings require cautious interpretation. In the case of Flash's detection, which detects at the entity level, labeling should have a more direct influence. This is reflected in the notably different results when using the labeling from [22], compared to ours or [20]. However, similar results are obtained when comparing our labeling with that of [20]. While these two labelings are indeed very close, the lack of distinction can be attributed to Flash's detection mechanism, which raises alerts not only on the directly detected entities but also on their 1-hop neighbors. This propagation effect can obscure the true impact of the labeling differences. For Kairos's detections, the total number of detections remains constant across the two labeling, but the specific intrusions detected vary, making the impact of labeling less visible under current detection metrics. In contrast, the influence of labeling is more evident in the results obtained with GAE. For instance, for client 501, we observe a drop in GAE's precision from 43% to 23% when switching label sets. As a consequence, nearly all activities are classified as either true positives or false positives for both labelings. This is due to our labeling including more activity during the night of September 24, whereas the other labeling includes only a subset of this period, resulting in different detection outcomes. Overall, while minor differences exist, our labeling and that of [20] show a high degree of agreement. Given that these labels were created by two separate teams of cybersecurity experts, this convergence suggests a strong consensus in identifying anomalous events of the dataset, further reinforcing the reliability of the proposed ground truth.

TABLE 2. IDS PERFORMANCES ON THE ORIGINAL ECAR LOGS DEPENDING ON THE LABELS.

Approach	Client	Labels	Precision	Recall	F1-score	AUC
Kairos [17]	051,201,501	from [20]	0.81	0.11	0.20	0.55
		Ours	0.81	0.10	0.18	0.55
Flash [22]	051	from [22]	0.14	0.85	0.24	x
		from [20]	0.98	0.23	0.38	x
		Ours	**0.99**	0.25	0.39	x
	201	from [22]	0.13	0.95	0.23	x
		from [20]	0.98	0.73	0.84	x
		Ours	**0.98**	0.73	0.84	x
	501	from [22]	0.79	0.92	0.85	x
		from [20]	1.00	0.72	0.83	x
		Ours	**1.00**	0.72	0.84	x
GAE [23] [24]	051	from [20]	0.07	1.0	0.14	0.31
		Ours	0.07	0.88	0.13	**0.34**
	201	from [20]	0.46	0.59	0.52	**0.72**
		Ours	0.14	0.67	0.24	0.69
	501	from [20]	0.23	1.0	0.38	0.30
		Ours	0.43	0.99	0.60	**0.42**

Table 3 shows that performances on the original and corrected versions of the dataset are similar, indicating that

the corrections did not significantly affect IDS behavior. However, most of the best results are achieved with the original version, particularly for the AUC and the F1-score. Among the three IDS, Flash appears to be less impacted by the dataset version and even performs slightly better with the corrected version, especially for client 501, where the recall and F1-score improve. In contrast, Kairos is more sensitive to these changes: with the corrected eCAR logs, it misclassifies two additional graphs as benign, resulting in two extra false negatives and a slight drop in recall and F1-score. However the corrected version provides a cleaner dataset structure, which makes the analysis and interpretation of results easier for security experts.

TABLE 3. IDS PERFORMANCES DEPENDING ON ECAR LOGS VERSION, WITH OUR LABELS.

Approach	Client	OpTC Version	Precision	Recall	F1-score	AUC
Kairos [17]	051,201,501	**original**	0.81	0.10	**0.18**	0.55
		corrected	0.79	0.09	0.16	0.54
Flash [22]	051	original	0.99	0.25	0.39	x
		corrected	1.0	0.25	0.39	x
	201	original	0.98	0.73	0.84	x
		corrected	0.99	0.73	0.84	x
	501	original	1.0	0.72	0.84	x
		corrected	1.0	0.78	**0.87**	x
GAE [23] [24]	051	**original**	0.07	0.88	0.13	**0.34**
		corrected	0.07	1.0	0.13	0.30
	201	**original**	0.14	0.67	0.24	**0.69**
		corrected	0.12	0.58	0.19	0.66
	501	**original**	0.43	0.99	0.60	**0.42**
		corrected	0.43	0.99	0.60	0.38

The results show that dataset labels have a major impact on IDS performance, whereas corrections to the eCAR logs have a smaller effect. This is likely because (1) most processes and events are unaffected by the errors (see Table 1), and (2) IDSs are trained and evaluated on the same version of the logs. For future work, we recommend using the corrected eCAR logs with a refined host-level label sets and our network-level labels for more reliable, reproducible, and interpretable evaluations.

6. Insights about dataset design

From the beginning of IDS evaluation, datasets and evaluation processes are being criticized [26]–[28]. However, recent work [29]–[37] shows that there are still many issues with newer datasets for NIDS: errors in the labels, errors in the configuration of the tools used for monitoring or launching the attacks, lack of diversity in the normal activity, etc. To our knowledge, only a few works [21], [38] examines carefully datasets dedicated to NIDS evaluation.

Building a dataset for intrusion detection requires significant efforts, so we encourage researchers to provide as much information as possible. This helps others identify errors and refine IDS by more easily analyzing alerts, especially false positives. As suggested in [39], a detailed network configuration should be provided. We recommend including the entire IT system configuration, such as firewall rules, routing, authentication processes, and services. Attack tool configurations, command lines, and monitoring setup (e.g., network capture points and log collection) should also be documented. The normal activity generation process should be clearly described and evaluated by dataset creators. A lack of diverse normal activity is a common flaw in IDS datasets [37]. Additionally, normal activity generation may introduce artifacts in logs or network captures, which researchers should identify and account for when designing and analyzing their IDS. One way to fulfill those recommendations is to use testbed generation tools [40], [41]. They enable the reproducibility of the dataset and give all the necessary data to understand how the dataset was created.

Our work may raise another question: as errors in a logging system can occur in real life, should they be fixed or not? For the DARPA OpTC dataset, the consequences on performances of IDS is not clear. However, we do not have any insights whether it is true or not for other datasets. Moreover, those errors may have less consequences on machine learning-based IDS as models are trained and evaluated on the same data but it can be more impactful on rule-based IDS.

7. Conclusion

In this article, we present two main contributions on the DARPA OpTC dataset. First, we identify and correct entity ID mismatches in the host logs that can compromise provenance integrity and affect IDS performance, using a dedicated detection and correction methodology.

The second main contribution of this work concerns the labeling of the dataset, essential for reliable evaluation given the lack of an official ground-truth. Recently, Nikulshin and Talhi [20] introduced a labeling for the host log data. In our work, we extend this effort by providing labels for both the network and host log data, and we compare our host-level labels with theirs to assess consistency. Experiment results show the impact of both the dataset corrections and the labeling on the performance of three state-of-the-art IDS. It appears that labeling has a strong influence on performance metrics, meaning that results published without clear labeling cannot be fully trusted. Moreover, the IDS performances obtained with the label sets of [20] and ours are very similar, indicating a strong consensus among different teams of cybersecurity experts. However, further harmonization between the two label sets could enhance consistency and support the development of standardized benchmarks. In contrast, the corrections applied to the logs have a smaller impact on IDS performances but stay interesting for an easier analyse and interpretation of results by security experts. We also gave some insights on how to design datasets and recommend the community to use testbed generation tool to provide all the information needed to reproduce and understand how the dataset was built.

8. Acknowledgement

We thank the Brittany Region and the Direction Générale de l'armement (DGA), in the context of CREACH LABS.

References

[1] A. Shiravi, H. Shiravi, M. Tavallaee, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," Computers & Security, vol. 31, no. 3, pp. 357–374, 2012.

[2] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in Proceedings of the 2015 Military Communications and Information Systems conference (MilCIS). IEEE, 2015, pp. 1–6.

[3] G. Maciá-Fernández, J. Camacho, R. Magán-Carrión, P. García-Teodoro, and R. Therón, "UGR '16: A new dataset for the evaluation of cyclostationarity-based network idss," Computers & Security, vol. 73, pp. 411–424, 2018.

[4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), vol. 1, 2018, pp. 108–116.

[5] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, "Flow-based benchmark data sets for intrusion detection," in Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS). ACPI South Oxfordshire, UK, 2017, pp. 361–369.

[6] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," Computers & Security, vol. 86, pp. 147–167, 2019.

[7] G. Creech and J. Hu, "Generation of a new IDS test dataset: Time to retire the KDD collection," in Proceedigns of the 2013 IEEE Wireless Communications and Networking conference (WCNC). IEEE, 2013, pp. 4487–4492.

[8] A. Hagberg, A. Kent, N. Lemons, and J. Neil, "Connected components and credential hopping in authentication graphs," in Proceedings of the 2014 International Conference on Signal-Image Technology and Internet-Based Systems (SITIS). IEEE Computer Society, Nov. 2014.

[9] W. Haider, G. Creech, Y. Xie, and J. Hu, "Windows based data sets for evaluation of robustness of host based intrusion detection systems (IDS) to zero-day and stealth attacks," Future Internet, vol. 8, no. 3, p. 29, 2016.

[10] DARPA, "DARPA transparent computing engagement 5," https://github.com/darpa-i2o/Transparent-Computing, 2019, [Online; accessed 09-April-2024].

[11] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, 2016, pp. 1035–1044.

[12] R. P. Lippmann, D. J. Fried, I. Graf, J. W. Haines, K. R. Kendall, D. McClung, D. Weber, S. E. Webster, D. Wyschogrod, R. K. Cunningham, and M. A. Zissman, "Evaluating intrusion detection systems: The 1998 DARPA off-line intrusion detection evaluation," in Proceedings of the DARPA Information Survivability Conference and Exposition (DISCEX'00), vol. 2. IEEE, 2000, pp. 12–26.

[13] R. P. Lippmann, J. W. Haines, D. J. Fried, J. Korba, and K. Das, "The 1999 DARPA off-line intrusion detection evaluation," Computer Networks, vol. 34, no. 4, pp. 579–595, 2000.

[14] DARPA, "Operationally transparent cyber dataset," https://github.com/FiveDirections/OpTC-data, 2019, online; accessed 09-April-2024.

[15] M. M. Anjum, S. Iqbal, and B. Hamelin, "ANUBIS: a provenance graph-based framework for advanced persistent threat detection," in Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, 2022, pp. 1684–1693.

[16] R. Paudel and H. H. Huang, "Pikachu: Temporal walk based dynamic graph embedding for network anomaly detection," in Proceedings of the 2022 IEEE/IFIP Network Operations and Management Symposium (NOMS). IEEE, 2022, pp. 1–7.

[17] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "Kairos: Practical intrusion detection and investigation using whole-system provenance," in Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P). IEEE, 2024, pp. 3533–3551.

[18] M. Zipperle, Y. Zhang, E. Chang, and T. Dillon, "PARGMF: A provenance-enabled automated rule generation and matching framework with multi-level attack description model," Journal of Information Security and Applications, vol. 81, p. 103682, 2024.

[19] S. T. King and P. M. Chen, "Backtracking intrusions," in Proceedings of the 19th ACM symposium on Operating Systems Principles, 2003, pp. 223–236.

[20] V. Nikulshin and C. Talhi, "Effective IDS under constraints of modern enterprise networks: revisiting the OpTC dataset," in Proceedings of the 2024 7th Conference on Cloud and Internet of Things (CIoT). IEEE, 2024, pp. 1–8.

[21] M. M. Anjum, S. Iqbal, and B. Hamelin, "Analyzing the usefulness of the DARPA OpTC dataset in cyber threat detection research," in Proceedings of the 26th ACM Symposium on Access Control Models and Technologies, 2021, pp. 27–32.

[22] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning," in Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P). IEEE Computer Society, 2024, pp. 139–139.

[23] K. Liu, Y. Dou, X. Ding, X. Hu, R. Zhang, H. Peng, L. Sun, and P. S. Yu, "PyGOD: A Python library for graph outlier detection," Journal of Machine Learning Research, vol. 25, no. 141, pp. 1–9, 2024. [Online]. Available: http://jmlr.org/papers/v25/23-0963.html

[24] K. Liu, Y. Dou, Y. Zhao, X. Ding, X. Hu, R. Zhang, K. Ding, C. Chen, H. Peng, K. Shu, L. Sun, J. Li, G. H. Chen, Z. Jia, and P. S. Yu, "BOND: Benchmarking unsupervised outlier node detection on static attributed graphs," in Proceedings of the Advances in Neural Information Processing Systems, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 021–27 035.

[25] "Wikipedia," https://en.wikipedia.org/wiki/Precision_and_recall, Accessed: 2024-01-20.

[26] J. McHugh, "Testing intrusion detection systems: a critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by lincoln laboratory," ACM Transactions on Information and System Security (TISSEC), vol. 3, no. 4, pp. 262–294, 2000.

[27] M. V. Mahoney and P. K. Chan, "An analysis of the 1999 DARPA/lincoln laboratory evaluation data for network anomaly detection," in Proceedings of the International Workshop on Recent Advances in Intrusion Detection. Springer, 2003, pp. 220–237.

[28] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proceedings of the 2009 IEEE symposium on computational intelligence for security and defense applications. Ieee, 2009, pp. 1–6.

[29] A. Kenyon, L. Deka, and D. Elizondo, "Are public intrusion datasets fit for purpose characterising the state of the art in intrusion event datasets," Computers & Security, vol. 99, p. 102022, 2020.

[30] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in Proceedings of the 2021 IEEE Security and Privacy Workshops (SPW). IEEE, 2021, pp. 7–12.

[31] M. Catillo, A. Del Vecchio, A. Pecchia, and U. Villano, "A critique on the use of machine learning on public datasets for intrusion detection," in Proceedings of the International Conference on the Quality of Information and Communications Technology (QUATIC). Springer, 2021, pp. 253–266.

[32] J. M. Peterson, J. L. Leevy, and T. M. Khoshgoftaar, "A review and analysis of the Bot-IoT dataset," in Proceedings of the 2021 IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE, 2021, pp. 20–27.

[33] A. Rosay, E. Cheval, F. Carlier, and P. Leroux, "Network intrusion detection: A comprehensive analysis of CIC-IDS2017," in Proceedings of the 8th International Conference on Information Systems Security and Privacy. SCITEPRESS-Science and Technology Publications, 2022, pp. 25–36.

[34] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and E. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in Proceedings of the International Conference on Risks and Security of Internet and Systems. Springer, 2022, pp. 18–33.

[35] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: a case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in Proceedings of the 2022 IEEE Conference on Communications and Network Security (CNS). IEEE, 2022, pp. 254–262.

[36] M. Catillo, A. Pecchia, and U. Villano, "Machine learning on public intrusion datasets: Academic hype or concrete advances in NIDS?" in Proceedings of the 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S). IEEE, 2023, pp. 132–136.

[37] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark nids datasets," in Proceedings of the 2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P). IEEE, 2024, pp. 658–675.

[38] J. Liu, M. A. Inam, A. Goyal, A. Riddle, K. Westfall, and A. Bates, "What we talk about when we talk about logs: Understanding the effects of dataset quality on endpoint threat detection research," in Proceedings of the 2025 IEEE Symposium on Security and Privacy (S&P). IEEE Computer Society, 2025, pp. 112–129.

[39] A. Gharib, I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "An evaluation framework for intrusion detection dataset," in Proceedings of the 2016 International conference on information science and security (ICISS). IEEE, 2016, pp. 1–6.

[40] R. Uetz, C. Hemminghaus, L. Hackländer, P. Schlipper, and M. Henze, "Reproducible and adaptable log data generation for sound cybersecurity experiments," in Proceedings of the Annual Computer Security Applications Conference (ACSAC), 2021, pp. 690–705.

[41] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger, and A. Rauber, "Maintainable log datasets for evaluation of intrusion detection systems," IEEE Transactions on Dependable and Secure Computing, vol. 20, no. 4, pp. 3466–3482, 2022.

Appendix A. Examples of multiple correction passes

Figure 4 shows a case where multiple corrections passes are needed. The first line represents the original logs timeline. In this timeline, processes $P1$, $P2$ and $P3$ are different `actorIDs` having the same `pid`. The two timeline errors are mixed together in this example: there is a timeline 2 error between $P1$ and $P2$ and a timeline 1 error between $P2$ and $P3$. It must be noted that $P2$ has no creation or termination events. We design our correction algorithm with some safeguards: notably we do not fix timeline 1 errors when there is no termination of one of the involved processes. After the detection step, our algorithm will not fix the timeline 1 error between $P2$ and $P3$ as $P2$ has no termination event but will fix the timeline 2 error between $P1$ and $P2$, replacing all instances of $P1$ by $P2$ leading to the second line of the figure. The detection step of the second pass will, this time, identify a timeline error 1 between $P2$ and $P3$ (on the second line of the figure). As $P2$ has a termination event, the correction step of the second pass will correct this error leading to the last line of Figure 4.

Appendix B. Differences between the labels for the two versions of the dataset

Table 4 presents the details of the differences of labels for the original version and the corrected version of the dataset. As explained in Section 2, we identify mismatches and overlaps in processes identifiers suggesting that some processes should not exist. As a results, the number of malicious `actorIDs` diminishes as the corrections for the timeline 2 merge some `actorIDs`. Because of the timeline errors identified, we choose to use the `pid` to label the malicious events. That explains why the corrections do not impact the number of events for our labeling.

TABLE 4. DIFFERENCES BETWEEN THE LABELS ON THE ORIGINAL AND CORRECTED DATASET

Scenario	# clients	OpTC version	# labeled actorIDs	# labeled events
SC1	17	corrected	752	124,453
		original	807	124,453
SC2	10	corrected	151	205,617
		original	175	205,617
SC3	2	corrected	118	30,855
		original	121	30,855

Appendix C. Differences between the labels of Nikulshin and Talhi and ours on the three clients used in the experiments

Figure 5 shows the difference in labels at the graph level (15-minutes graphs) for the three hosts used in the experiments. For Host 051, the labels are very close: the only difference is in the labels of the last two graphs that we do not label as malicious contrary to Nikulshin and Talhi. There are bigger differences for Host 201 and Host 501. For Host 201, Nikulshin and Talhi labeled as malicious some activities not described in the PDF file describing the Red Team's activity. Indeed, it appears that some malicious executables are launched repeatedly in the afternoon of the attack day, as well as on the 24th and 25th of September, suggesting that the Red Team may have missed the cleanup after their initial compromise. We did not label this activity as it is not described in the PDF file but it shall be labeled as malicious. For Host 501, the agent was running the

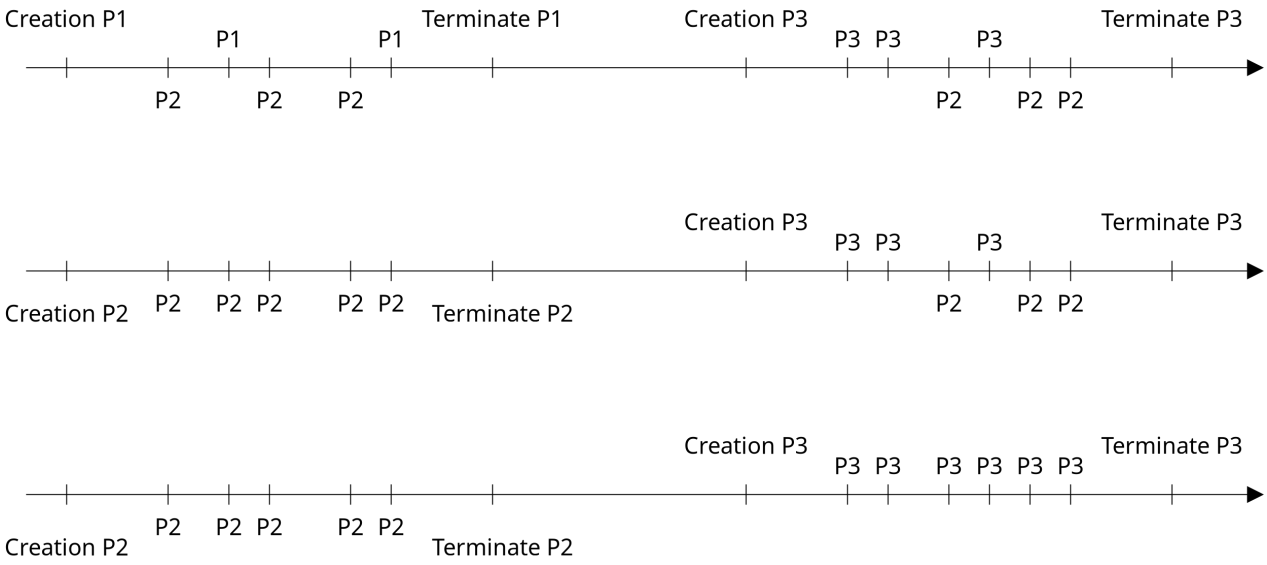
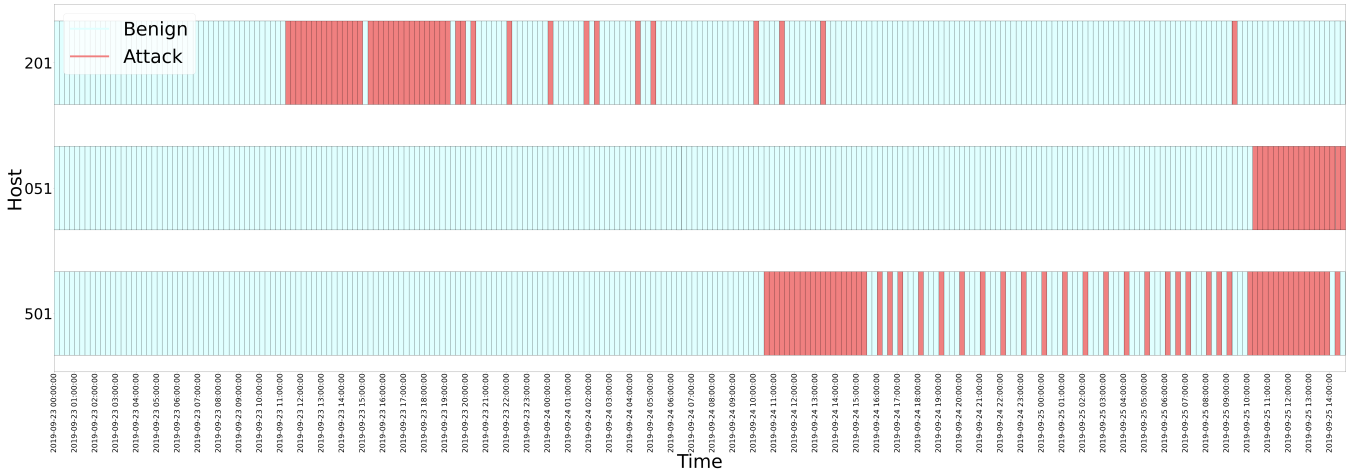
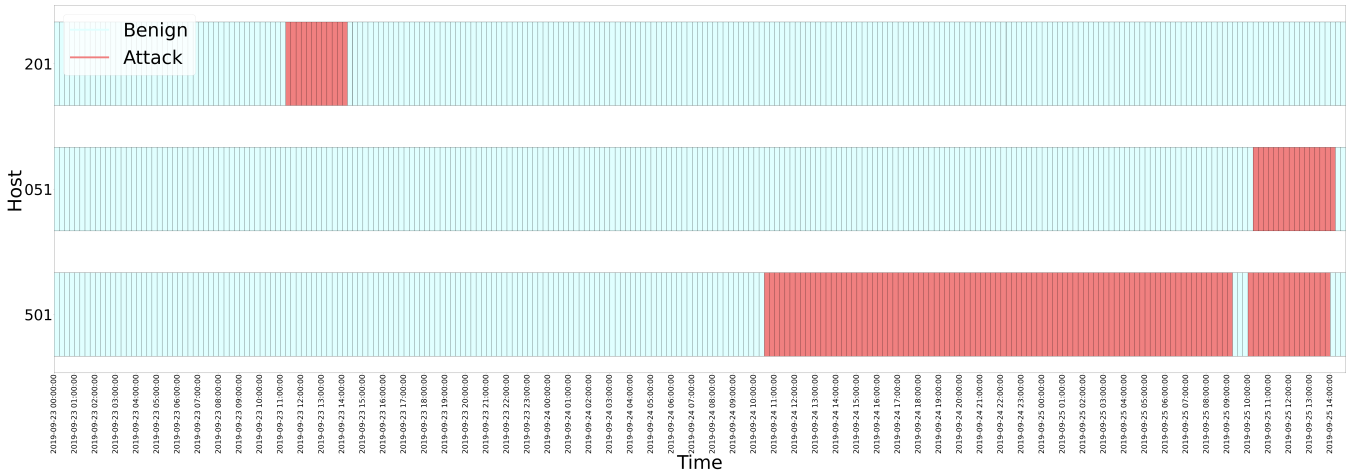


Figure 4. Multiple correction passes

whole night of the 24th of September and contacted the C&C server all nightlong. Our methodology labeled all the graphs of the night as malicious, while Nikulshin and Talhi labeled only some parts of the activity. We did not dive into their labeling script to understand why it misses most of the beaconing of the agent during the night.



(a) Labels from Nikulshin and Talhi [20]



(b) Our Labels

Figure 5. Visualization of different labels, for 15-minutes graphs detection, for hosts 051, 201 and 501.